

VacTracker Technical Cheat Sheet

*Prepared for your supervisor meeting. Everything below is based only on the actual code. Where the code does not support a claim, it says **Note / Not implemented**.*

1. FRONTEND

1.1 Flutter

- **Name:** Flutter — Google's open-source UI toolkit for building one app for Android, iOS, web, Windows and macOS from **one** codebase.
- **Why use it:** ideal for a cross-platform farm app (farmers on Android + web); write the UI once, run everywhere.
- **Where used:** the whole app is under `frontend/lib`; entry point `main.dart` (`runApp(VacTrackerApp())`); screens in `frontend/lib/screens/`.
- **Example:** `main.dart` line 309: `runApp(const VacTrackerApp())`.
- **Supervisor line:** "I built the app with Flutter so the same code runs on Android and the vet's device using one UI framework."

1.2 Dart

- **Name:** Dart — the language Flutter apps are written in.
- **Why:** it is Flutter's language; all models, screens and services are Dart.
- **Where used:** every `.dart` file under `frontend/lib`.
- **Example:** `models/flock.dart` defines a `Flock` class with `fromJson`.
- **Supervisor line:** "Dart is the typed language behind Flutter."

1.3 GoRouter

- **Name:** GoRouter — a routing library that maps URL-style paths to screens.
- **Why:** declarative navigation and easy data passing via path parameters / `extra`.
- **Where used:** router defined in `main.dart` (`GoRouter(...)`), wired via `MaterialApp.router(routerConfig: _router)`; screens navigate with `context.push('/vaccine-library/$lang')`.
- **Example:** `main.dart`: `GoRoute(path: '/login-otp', ...)`.
- **Supervisor line:** "GoRouter is my route table — every screen has a path like `/login/farmer/en` and GoRouter swaps the screen."

1.4 Dio

- **Name:** Dio — a powerful Dart HTTP client with interceptors, timeouts and auth headers.
- **Why:** used for authenticated calls and to inject the JWT on every request via an interceptor.
- **Where used:** only in `auth_service.dart` — `final Dio dio = Dio(BaseOptions(baseUrl: ApiConfig.baseUrl))`; plus `InterceptorsWrapper.onRequest` adding `Authorization: Bearer <token>`.

- **Supervisor line:** "Dio is my client for login/OTP; it auto-attaches the JWT and clears login data on 401."

1.5 HTTP package

- **Name:** the standard `package:http/http.dart`.
- **Why:** most data calls use this simple built-in client.
- **Where used:** flock, vaccination, reminder, vaccine, notification, sick-report, vet-dashboard, farmer-detail and vaccine-library services (and directly in `add_flock_page.dart`).
- **Supervisor line:** "I use Flutter's standard `http` for most JSON API calls."

1.6 SharedPreferences

- **Name:** SharedPreferences — Flutter's local key-value storage.
- **Why:** persist the login token and user name/id/role/profile image across restarts.
- **Where used:** `services/storage_service.dart` (saveToken, getToken, saveUser, getName, clearAll); key `"access_token"`.
- **Supervisor line:** "I store the login token in SharedPreferences so the app remembers the user — but it isn't encrypted, so it's a security fix."

1.7 flutter_secure_storage

- **What:** OS-level secure storage (Keychain/Keystore).
- **Why declared:** the plan was to store the JWT securely.
- **Where used: not used in frontend/lib** — `StorageService` uses SharedPreferences.
- **Supervisor line:** "I added the secure-storage package but haven't wired it in yet; it's on my fix list."

1.8 StatefulWidget

- **Name:** a widget that holds mutable state and can rebuild.
- **Why:** data-loading screens need state (isLoading / error / results).
- **Where used:** most screens (e.g. `vaccination_history.dart`, `add_flock_page.dart`, `login_otp_screen.dart`).
- **Supervisor line:** "Data screens are StatefulWidget so they can redraw when the API responds."

1.9 setState

- **What:** tells Flutter "state changed, redraw".
- **Where:** `login_otp_screen.dart` `setState(() { loading = true; })`, `add_flock_page.dart` `setState(() { _selectedBreed = id; })`.
- **Supervisor line:** "I wrap UI updates in `setState` so Flutter rebuilds the screen."

1.10 Image Picker

- **Status:** declared in `pubspec.yaml` and registered as a plugin, but **not used in frontend/lib**. → **Not implemented.**

1.11 PDF generation / sharing

- **Name:** `vaccination_pdf_service.dart` using `path_provider` + `share_plus`.

- **How it works:** `vaccination_history_page.dart` calls `exportHistoryPdf(...)`; the service writes a **plain-text file** with a `.pdf` extension via `file.writeAsString(content)`, saves it to the app documents folder and shares via `SharePlus.instance.share`.
 - **Limitation:** it is a text file named `.pdf`, **not** a real PDF (no `package:pdf`).
 - **Supervisor line:** "I implemented share-to-PDF for the vaccination report — it works but produces plain text, so a real PDF library is a future improvement."
-

2. BACKEND

2.1 NestJS

- **What:** a structured, TypeScript-first Node.js framework for APIs with dependency injection and modules/controllers/services.
- **Why:** keeps a growing API (users, flocks, vaccines, vaccinations, reminders, sick reports, vet dashboard) organised.
- **Where used:** everything in `backend/src`. Bootstrap: `main.ts` creates the app, enables CORS, registers a global `ValidationPipe`, listens on port 3100.
- **Example:** `backend/src/app.module.ts` imports all feature modules.
- **Supervisor line:** "NestJS is my web framework on Node — it organises the API into modules, controllers and services with automatic dependency injection."

2.2 TypeScript

- **What:** a typed superset of JavaScript (types, interfaces, enums).
- **Why:** catches bugs at compile time; easier to maintain.
- **Where:** every backend file is `.ts` (typescript ^5.7.3 in `package.json`).
- **Example:** `user.entity.ts` uses `enum UserRole { FARMER=..., VETERINARIAN=... }`.
- **Supervisor line:** "I wrote the backend in TypeScript so the compiler catches type mistakes before they run."

2.3 Controller

- **What:** the HTTP layer — "listens" for requests on a route, reads them, calls the service.
- **Where:** every feature has one, e.g. `backend/src/flocks/flocks.controller.ts`.
- **Example:** `@Post() create(@Body() dto, @Request() req) with @UseGuards(JwtAuthGuard)`.
- **Supervisor line:** "A controller is the API's front door — it decides which URL/method maps to which action."

2.4 Service

- **What:** the business-logic layer — the actual rules and queries.
- **Where:** `flocks.service.ts`, `vaccinations.service.ts`, `sick-reports.service.ts`, `users.service.ts`, `vet-dashboard.service.ts`.
- **Example:** `vaccinations.service.ts calculateVaccinationStatus()` decides overdue/due-soon/on-time.
- **Supervisor line:** "The service holds the business rules; the controller just forwards the request."

2.5 Module

- **What:** a NestJS grouping tying a controller, its services and entities together.
- **Where:** `app.module.ts` imports all; each feature folder has its own module (e.g. `flocks.module.ts`).
- **Supervisor line:** "A module is a small box that groups one feature — controller, service and DB entity bindings."

2.6 DTO (Data Transfer Object)

- **What:** a plain class defining the shape/validation of data coming into an endpoint.
- **Where:** `flocks/dto/create-flock.dto.ts`, `vaccinations/dto/create-vaccination.dto.ts`.
- **Example:** `create-flock.dto.ts` uses `@IsString()`, `@IsInt()`, `@IsDateString()`.
- **Supervisor line:** "A DTO is the contract of what an endpoint accepts; NestJS validates the request against it."

2.7 class-validator

- **What:** a validation library using decorators to check payloads.
- **Where:** enabled globally (`new ValidationPipe({ whitelist: true, transform: true })`) in `main.ts`; used in DTO files.
- **Supervisor line:** "class-validator checks incoming data — invalid requests are rejected before reaching my logic."

2.8 class-transformer

- **What:** converts plain JSON into typed classes (enables `transform: true`).
- **Where:** powers the global `ValidationPipe` in `main.ts`.
- **Supervisor line:** "class-transformer turns raw JSON into typed objects so validation works."

2.9 TypeORM

- **What:** an ORM mapping TypeScript classes to database tables.
- **Why:** I write entities/relationships in code; it can create the schema automatically.
- **Where:** entities in every `entities/` folder; configured in `app.module.ts` (`TypeOrmModule.forRootAsync({ type: 'postgres' })`).
- **Example:** `@Entity('users')` class `User` with `@PrimaryGeneratedColumn()`, `@OneToMany(...)`.
- **Supervisor line:** "TypeORM is my DB connector — each class is a table and decorators define columns and relationships without SQL."

2.10 Repository

- **What:** TypeORM's per-entity query object (`.find`, `.findOne`, `.save`, `.create`, `.remove`).
- **Where:** injected into services, e.g. `@InjectRepository(Flock) private flockRepository: Repository<Flock>`.
- **Example:** `flockRepository.findOne({ where: { flock_id: id, farmer: { phone } })`.
- **Supervisor line:** "A repository is the service for one table — find/save/delete methods without writing SQL."

2.11 PostgreSQL

- **What:** a free, open-source relational database.
- **Why:** reliable, SQL-standard, supports enums/JSON.
- **Where:** all entities persist here; config in `app.module.ts` + `backend/.env` (`DB_DATABASE=vactracker`).
- **Supervisor line:** "We chose PostgreSQL for reliability and strong data-integrity features (enums, foreign keys, JSON)."

2.12 pg

- **What:** the low-level Node driver PostgreSQL.
- **Why:** TypeORM uses `pg` internally; listed in `package.json` ("`pg`": "`^8.22.0`").
- **Supervisor line:** "`pg` is the driver TypeORM uses to talk to Postgres; I don't call it directly."

2.13 @nestjs/config

- **What:** reads environment variables from `.env`.
- **Where:** `ConfigModule.forRoot({ isGlobal: true })` in `app.module.ts`; `ConfigService` feeds the DB connection; `JWT_SECRET` read in `auth.module.ts/jwt.strategy.ts`.
- **Supervisor line:** "`@nestjs/config` loads my `.env` settings so credentials aren't hard-coded."

3. AUTHENTICATION — COMPLETE FLOW

The flow, step by step (server: `backend/src/auth/auth.service.ts` + `auth.controller.ts`; frontend: `auth_service.dart`, `login_screen.dart`, `login_otp_screen.dart`, `storage_service.dart`):

1. **Phone number entered** — on `LoginScreen` the farmer enters their phone.
2. **Check phone** — `AuthService.checkPhone(phone) → userRepository.findOne({ where: { phone } })`, returns `{ exists: boolean }`. Route `POST /auth/check-phone`.
3. **Send OTP** — `sendOtp(phone)` only allows registered users. Development-only: OTP is **hard-coded** `123456`, hashed with `bcrypt.hash(otp, 10)`, saved as an `OtpCode` (phone, `code_hash`, `attempts=0`, `expires_at = now + 5 min`), `console.log'd`, and **returned in the HTTP response** (`// TODO: REMOVE OTP FROM API RESPONSE BEFORE PRODUCTION`). Route `POST /auth/send-otp`.
4. **Verify OTP** — `verifyOtp(phone, otp)` loads the latest `OtpCode` for the phone, does `bcrypt.compare(otp, code_hash)`.
5. **Generate JWT** — on success it does `this.jwtService.sign({ user_id, phone })` via `@nestjs/jwt` (`expiresIn: '7d'; secret JWT_SECRET || 'vactracker_secret'` — `auth.module.ts`). Returns `{ message, access_token, user }`.
6. **Store JWT** — `login_otp_screen.dart` await `StorageService.saveToken(result["access_token"])` then `saveUser(user)`, writing to **SharedPreferences** key `"access_token"` (`storage_service.dart`).
7. **Send JWT with requests** — (a) Dio interceptor in `auth_service.dart` auto-adds `Authorization: Bearer $token`; (b) every http service reads `StorageService.getToken()` and sets the header manually.
8. **JwtAuthGuard** — `jwt-auth.guard.ts` extends `AuthGuard('jwt')`, applied via `@UseGuards(JwtAuthGuard)` on controllers/methods. It runs the Passport strategy before the request proceeds.

9. **JwtStrategy** — `jwt.strategy.ts` (passport-jwt) extracts the token from the header, verifies signature/expiry with `secretOrKey`, then `validate(payload)` returns the payload.
10. **req.user** — Passport attaches the payload to the request; endpoints read `req.user.user_id` / `req.user.phone` / `req.user.role` (e.g. `vaccinations.controller.ts`).

Security notes on the current flow:

- OTP is a fixed dev value and is returned by the API. **Not production-ready.**
- JWT payload has **no role claim**, so `req.user.role` is `undefined` (breaks role-based logic).
- Hard-coded JWT secret fallback.
- `expires_at` is stored but `verifyOtp` **never checks it**; attempts counter is stored but not enforced.
- `SmsService` (`sms.service.ts`) is an **empty stub** — no real SMS is sent. **Not implemented.**

Executable 1-2 sentences

"Auth is phone + OTP: check the number, send an OTP (bcrypt-hashed), verify it, then issue a JWT containing user_id and phone. The app stores it, sends it in the Authorization header, and a Passport JWT strategy verifies it and puts the identity on req.user. Today the OTP is still a dev-only fixed code and the JWT sits in plain SharedPreferences."

4. DATABASE — ENTITIES AND RELATIONSHIPS

Stack: PostgreSQL via TypeORM. Schema is auto-created each boot because `app.module.ts` sets `synchronize: true` and `autoLoadEntities: true` (⚠ never use `synchronize: true` in production).

Entity map (real names from code)

```

user (users) 1 — * flock (flocks)      (OneToMany: user.flocks )
user 1 — * sick_report (sick_reports)  (OneToMany: user.sickReports)
user 1 — * vet_farmer_connection (vet/farmer both directions)
flock 1 — * vaccination 1 — * reminder (vaccination → reminder)
vaccination * — 1 vaccine      (ManyToOne: vaccination.vaccine)
flock 1 — * sick_report
vaccine_library * — 0..1 vaccine (ManyToOne nullable)
user 1 — * notification
user 1 — * reminder (farmer)
otp_code (standalone auth table)

```

User (`users/entities/user.entity.ts`) — `@Entity('users')` `user_id` (PK), `name`, `phone` (unique), `password_hash` (nullable), `role` (enum `farmer/veterinarian`), `village`, `province`, `language_pref` (enum `km/en`), `profile_image_url`, `share_code` (unique UUID used by vets to connect), `created_at`, `updated_at`. Relations: `@OneToMany flocks` (to `Flock.farmer`), `@OneToMany sickReports` (to `SickReport.reporter`), `@OneToMany vetConnections + farmerConnections` (to `VetFarmerConnection`).

Flock (`flocks/entities/flock.entity.ts`) — `@Entity('flocks')` `flock_id` (PK), `farmer` (`@ManyToOne` → `User`, FK column `farmer_id`), `batch_name`, `bird_count`, `breed`, `age`, `age_unit`, `date_acquired`,

health_status, created_at, updated_at. Relations: @OneToMany sickReports.

Vaccination (vaccinations/entities/vaccination.entity.ts) — @Entity('vaccinations') + @Index(['flock', 'vaccine', 'date_given']) vaccination_id (PK), flock (@ManyToOne → Flock, FK flock_id), vaccine (@ManyToOne → Vaccine, FK vaccine_id), administered_by (@ManyToOne → User, FK administered_by), date_given, next_due_date (nullable), status (enum on_time/due_soon/overdue/completed), photo_url, created_at.

Vaccine (vaccines/entities/vaccine.entity.ts) — @Entity('vaccines') vaccine_id (PK), name_en, name_km, disease_en, disease_km, interval_days (nullable integer — drives automatic next-due-date), notes_en/notes_km.

VaccineLibrary (vaccine-library/entities/vaccine-library.entity.ts) — @Entity('vaccine_library') library_id (PK), optional vaccine_id (@ManyToOne → Vaccine, onDelete: 'SET NULL'), dual-language fields (name_*, disease_*, description_*, vaccine_description_*, why_important_*, usage_*, precautions_*, other_info_*), category, created/updated. Seeded on startup by vaccine-library.seeder.ts.

Vaccination → Reminder (reminders/entities/reminder.entity.ts) — @Entity('reminders') reminder_id (PK), vaccination (@ManyToOne → Vaccination, FK vaccination_id, onDelete: 'CASCADE'), farmer (@ManyToOne → User, FK farmer_id, onDelete: 'CASCADE'), title, message, scheduled_date, sent_at (nullable), status (enum pending/sent/failed/completed), sent_by (enum system/vet), created_at.

SickReport (sick-reports/entities/sick-report.entity.ts) — @Entity('sick_reports') report_id (PK), flock_id (@ManyToOne → Flock + column), reported_by (@ManyToOne → User.reporter), reportType (enum: disease/injury/death/other), affectedCount, symptoms, photoUrl, reportDate, status (enum pending/reviewed/resolved), plus vet-response fields: vet_id (@ManyToOne → User), vetNotes, vetDiagnosis, vetAdvice, recommendedAction (enum), followUpDate, respondedAt.

VetFarmerConnection (users/entities/vet-farmer-connection.entity.ts) connection_id (PK), vet_id (@ManyToOne → User), farmer_id (@ManyToOne → User), status (enum pending/accepted/rejected), created_at. Farmers add a vet with their share_code (no approval — set ACCEPTED immediately).

Notification (notifications/entities/notification.entity.ts) notification_id (PK), farmer_id (@ManyToOne → User, CASCADE), title, message, type (enum vet_response/system/vaccination_overdue), reference_id, is_read, data (JSON), created_at.

OtpCode (auth/entities/otp-code.entity.ts) — @Entity('otp_codes') otp_id (PK), phone, code_hash (bcrypt), attempts, expires_at, created_at.

Database concepts with real examples

- **Primary key (PK):** unique id per row. Real: @PrimaryGeneratedColumn() on user_id, flock_id, vaccination_id, etc. → auto-incrementing integers.
- **Foreign key (FK):** a column referencing another table's PK to link rows. Real: @JoinColumn({ name: 'farmer_id' }) in flock.entity.ts, flock_id/vaccine_id in vaccination.entity.ts.
- **UUID:** a globally-unique identifier string. Real: share_code is generated with randomUUID() in users.service.ts (not used as PK).

- **One-to-many:** one parent has many children. Real: `User.flocks (@OneToMany)`, `User.sickReports`, `Flock.sickReports`.
- **Many-to-one:** many children point to one parent — the inverse side. Real: `Flock.farmer (@ManyToOne)`, `Vaccination.flock/vaccine`.
- **Enum:** a fixed set of allowed values stored as a column type. Real: `role`, `language_pref`, `vaccination.status`, `report.status`, `ReminderStatus`, `NotificationType`.

5. API — HOW A FEATURE TRAVELS THROUGH THE SYSTEM

The general journey for any feature: Flutter screen → Flutter service (HTTP call) → NestJS controller → NestJS service → TypeORM repository → PostgreSQL → JSON response → Flutter UI (parsed into models, rendered with `setState`).

Example 1 — Flock management (the walk-through)

1. **Flutter screen** `farmer_dashboard_page.dart` (or `add_flock_page.dart`) triggers a call.
2. **Flutter service** `flock_service.dart` reads the JWT (`StorageService.getToken()`), builds the URL `${ApiConfig.baseUrl}/flocks`, calls `http.get(...).timeout(8s)`.
3. **HTTP request** `GET /flocks` with `Authorization: Bearer <token>`.
4. **NestJS controller** `flocks.controller.ts @Get()` `findAll(@Request() req)` with `@UseGuards(JwtAuthGuard)`; guard resolves the user.
5. **Service** `flocks.service.ts findAll(phone)` looks up the farmer, then `flockRepository.find({ where: { farmer: { user_id } }, relations: { farmer: true }, order: { created_at: 'DESC' } })`.
6. **Repository** TypeORM turns it into a SQL `SELECT` and returns rows.
7. **PostgreSQL** executes the query against the `flocks` table.
8. **Response** JSON array of flocks.
9. **Flutter UI** `Flock.fromJson` maps each item; `setState` rebuilds the list.

Example 2 — Add flock

screen `add_flock_page.dart` → `POST ${baseUrl}/flocks` (posted via `http.post` with token) → `flocks.controller.ts @Post()` `create(@Body() dto: CreateFlockDto, @Request() req)` (guarded) → `flocks.service.ts create(dto, phone)` finds the farmer, `flockRepository.create({ ...dto, farmer })` then `.save()` → Postgres insert → returns the new `Flock`.

Example 3 — Log a vaccination

→ `log_vaccination_step*.dart` screens collect data → `vaccination_service.dart createVaccination(data)` → `POST /vaccinations` → `vaccinations.controller.ts @Post()` → `vaccinations.service.ts create(dto, phone)`: verifies user, loads flock + verifies the flock belongs to user (else `NotFoundException`), loads vaccine, de-dupes (prevents double submit), **computes** `next_due_date` (user-provided OR `date_given + interval_days`), **computes status**, saves the row, then marks old reminders/notifications done and creates a new `Reminder` → returns the saved `Vaccination`.

Example 4 — View vaccination history

→ `vaccination_history_page.dart` (StatefulWidget) → `vaccination_service.dart`
`fetchVaccinationsByFlock(flockId)` → GET `/vaccinations/flock/:id` →
`vaccinations.controller.ts @Get('/flock/:id')` → `vaccinations.service.ts`
`findByFlock(flockId, phone)` verifies the flock belongs to the user, `vaccinationRepository.find({`
`where: { flock: { flock_id } }, relations: { flock, vaccine, administered_by }, order: {`
`date_given: 'DESC' } })` → returns list → screen renders records (and can share a PDF report).

Example 5 — Get vaccine list

→ `vaccine_service.dart` `fetchVaccines()` → GET `/vaccines` (**public, no token**) →
`vaccines.controller.ts @Get()` `findAll()` (no guard) → `vaccines.service.ts` `findAll()` →
`vaccineRepository.find()` → list → `Vaccine.fromJson`.

Example 6 — Get reminders

→ `reminder_service.dart` `fetchMyReminders()` → GET `/reminders/me` (guarded) →
`reminders.controller.ts @Get('me')` → `reminders.service.ts` `findByFarmer(req.user.user_id)`
 — first `syncRemindersForFarmer()` (creates any missing) then a query builder that excludes **COMPLETED** and
 sorts **PENDING** first, pending-date ascending.

Example 7 — Sick report

→ `sick_report.dart` screen → `sick_report_service.dart` → POST `/sick-reports` (guarded) → `sick-`
`reports.controller.ts @Post()` → `sick-reports.service.ts` `create(dto, userId)` stamps
`reportedBy: userId`, saves, returns the report.

Example 8 — Vet dashboard → `vet_dashboard_page.dart` → `vet_dashboard_service.dart`
`getDashboardStats()` → GET `/vet/dashboard/stats` (guarded) → `vet-dashboard.controller.ts` →
`vet-dashboard.service.ts` `getDashboardStats(vetPhone)`: finds the connected farmers (accepted
 connections), their flocks, the vet's own vaccinations and the sick reports; computes summary cards
 (connectedFarmers, totalFlocks, newSickReports, overdueVaccinations) and a per-farmer status (healthy / sick /
 overdue / due_soon), then sorts by priority. Returns the stats to the screen.

All 7 features above exist and are wired end-to-end.

6. REMINDER SYSTEM

Files: `backend/src/reminders/reminder.scheduler.ts`, `reminders.service.ts`,
`reminders.controller.ts`; entity `reminders/entities/reminder.entity.ts`; scheduler from
`@nestjs/schedule`, registered with `ScheduleModule.forRoot()` in `app.module.ts`.

1. When a vaccination is created — in `vaccinations.service.ts` `create()`, after saving the new
 vaccination it:

- calls `remindersService.markCompleted(who)` via `mark-reminders-completed`
 (`reminders.service.ts` `markRemindersCompleted`) so old pending reminders for that flock are set

COMPLETED,

- clears overdue notifications for that flock (`notificationsService.markOverdueNotificationsRead`),
- and if the new vaccination has a `next_due_date`, calls `createReminder(...)` right away (`reminders.service.ts`).

2. How `next_due_date` is calculated — three rules in `vaccinations.service.ts` `create()`:

- if the client sent `next_due_date`, use it;
- else if `vaccine.interval_days > 0`, `next_due_date = date_given + interval_days`;
- otherwise it is `null` (no reminder).

3. How `status` is determined — `calculateVaccinationStatus(nextDueDate)` in `vaccinations.service.ts`:

- no `next_due_date` → `ON_TIME`;
- due date in the past → `OVERDUE`;
- within 3 days → `DUE_SOON`;
- otherwise → `ON_TIME`. (`COMPLETED` exists in the enum but the create flow never sets a vaccination to that value.)

4. How Cron works — `@nestjs/schedule` + `ScheduleModule.forRoot()`; a method annotated with `@Cron('* * * * *')` is invoked on the given schedule.

5. Why `@Cron('* * * * *')` — that expression means **run every minute**. It is a simple/demo choice so reminders are generated quickly after a vaccination's due date or overdue. Production would use a longer cadence.

6. How the scheduler finds due vaccinations — `reminder.scheduler.ts` `generateReminders()`:

- computes `tomorrow = today + 1 day` (formatted `YYYY-MM-DD`),
- queries `vaccination.next_due_date = tomorrow` (with joins `flock` → `farmer` to get the owner),
- logs the count and calls `createReminderIfNotExists` for each.

The other method `generateOverdueNotifications()` queries `next_due_date < today` and `status != COMPLETED`, and for each calls `notificationsService.createOverdueVaccinationNotification(...)` (writes an in-app `Notification`, not a `Reminder`). **7. How Reminder records are created** — `createReminderIfNotExists(vaccination)`:

- if no `next_due_date` or no owner → return;
- checks for an existing reminder with the same `vaccination_id + farmer_id + scheduled_date` (dedupe to avoid duplicates);
- otherwise inserts { `vaccination_id`, `farmer_id`, `title: 'Vaccination Reminder'`, `message: '...<vaccine name> is due tomorrow for <batch_name>.'`, `scheduled_date: next_due_date`, `status: PENDING`, `sent_by: SYSTEM` }. A second path, `reminders.service.ts` `syncRemindersForFarmer`, scans a farmer's vaccinations on request and creates any missing reminders lazily.

8. How Flutter retrieves reminders — `reminder_service.dart` `fetchMyReminders()` → GET `/reminders/me` → `reminders.service.ts` `findByFarmer(farmerId)`:

- calls `syncRemindersForFarmer` first (lazy creation),
- then queries that farmer's reminders, **excluding COMPLETED**, joining vaccination→flock→vaccine, sorted PENDING-first then `scheduled_date` ascending.

What is incomplete / only partially implemented

- **Actual reminder delivery is NOT implemented.** Reminders are created with `status: PENDING` but nothing ever sends an SMS/notification or sets `status` to `SENT/FAILED`. `SmsService` (`sms.service.ts`) is an **empty stub**.
- The in-app `notifications` table is a separate mechanism (vet responses + overdue alerts) — it is not the reminder scheduler's delivery channel.
- `sent_by` enum allows `vet`, but **no vet-created-reminder path** exists in the code.
- The overdue logic creates `Notification` rows via the cron and again lazily on app load (`notifications.service.ts syncOverdueNotifications`), not `Reminder` rows.
- Reminders depend on the **NestJS process being alive**; reminders are generated in-memory from the DB at runtime, with no queued background job.
- Two cron methods run every minute (due-tomorrow reminders + overdue notifications). That is heavy but works for a demo.

7. SECURITY ISSUES FOUND

Format: **Problem** → **Why** → **Fix** → **Priority**.

7.1 JWT stored in plain SharedPreferences

- Problem: the lifetime JWT is saved with SharedPreferences (`storage_service.dart` key `"access_token"`), plaintext.
- Why: it is extractable from device storage/backup; malicious use for 7 days.
- Fix: store in `flutter_secure_storage` (Keychain/Keystore).
- Priority: ● Must fix.

7.2 Public / unprotected endpoints

- Problem: `GET /vaccines`, `GET /vaccines/:id` and `GET /vaccine-library` (+ detail) have no `@UseGuards` (see `vaccines.controller.ts`, `vaccine-library.controller.ts`).
- Why: public reads of reference data are partly by design, but it is inconsistent and any future write endpoint could be missed.
- Fix: decide the intended public set; keep all writes guarded and add role checks.
- Priority: ● Should fix.

7.3 `GET /users` leaks password hashes

- Problem: `users.service.ts findAll()` returns the full `User` entity including `password_hash` (only filtered to the requester's own id at the controller).
- Why: leaked bcrypt hashes can be offline-cracked; also `GET /users` returns the current user only.
- Fix: exclude `password_hash` from responses (DTO or `select: false`).
- Priority: ● Must fix.

7.4 Vaccination endpoints

- Status: guarded at class level and services do ownership checks on `findOne/findByFlock/create`. Good.
- Remaining risk: no role splitting (any authenticated user can log vaccinations for their own flocks, acceptable for v0) and no rate limiting.
- Priority: 🟡 Should fix (add rate limiting later).

7.5 Vaccine endpoints (write access)

- Problem: `POST/PATCH/DELETE /vaccines` are guarded but ANY authenticated user can run them.
- Why: a farmer could corrupt the shared vaccine reference list.
- Fix: restrict writes to a vet/admin role.
- Priority: 🔴 Must fix.

7.6 Authorization headers / single client

- Problem: token header logic is duplicated across Dio (`auth_service.dart`) and every `http` service.
- Why: risk of forgetting headers, inconsistent error handling.
- Fine: a single Dio helper / interceptors for everything.
- Priority: 🟡 Should fix.

7.7 Password / OTP hashing

- Problem (password): password is bcrypt-hashed with 10 rounds — **this is good** and correct in `users.service.ts`.
- Problem (OTP): the OTP is the **fixed constant 123456** and is **returned in the API JSON response** (`auth.service.ts sendOtp()`).
- Why: anyone who calls the endpoint can read it; a fixed code never rotates; that defeats OTP's purpose.
- Fix: generate a random OTP, send it out-of-band (SMS), do **not** return it, and check `expires_at` + max attempts.
- Priority: 🔴 Must fix.

7.8 Role-based access

- Problem: the JWT signed in `auth.service.ts` is `{ user_id, phone }` — **it has no role**. Yet several controllers read `req.user.role` (users, sick-reports) and vet routes don't enforce role with a guard.
- Why: role conditions silently evaluate to `undefined`; vet-only data could be reachable by a farmer with a valid token.
- Fix: add `role` (+ `name`) to the JWT and a real `RolesGuard`.
- Priority: 🔴 Must fix.

7.9 Hard-coded JWT secret

- Problem: `jwt.strategy.ts / auth.module.ts` use `process.env.JWT_SECRET || 'vactracker_secret'`.
- Why: if `.env` missing, the secret is public and anyone could forge tokens.
- Fix: require `JWT_SECRET`, fail fast if absent.
- Priority: 🟡 Should fix.

7.10 Other hardening gaps

- No brute-force/rate limiting on `send-otp` / `verify-otp`.
- `expires_at` stored but **not enforced** in `verifyOtp`; `attempts` stored but not limited.
- `owner`-ownership lookup patterns exist (good) but are inconsistent across features (some use `throw new Error` returning 500, e.g. `vet-dashboard.service.ts`, `farmer-detail.service.ts`).
- CORS is wide open (`app.enableCors()` with no origin restriction).

8. CODE QUALITY REVIEW

● Must Fix Before Testing

1. **JWT stored in plaintext `SharedPreferences`** instead of `flutter_secure_storage` (frontend `storage_service.dart`).
2. **GET `/users` returns `password_hash`** (`users.service.ts` `findAll`).
3. **Fixed `123456` OTP returned to client** (`auth.service.ts` `sendOtp`) — no expiration check, no attempt limit.
4. **No real role enforcement** — JWT has no `role`; `vet/admin` routes not guarded by role (`sick-reports` update logic trusts `role`; `vaccine` write endpoints open to any logged-in user).
5. **synchronize: `true`** in production (`app.module.ts`) — can drop/alter data unexpectedly; use migrations.

● Should Fix

6. **Two HTTP clients** — `Dio` (auth only) + `http` everywhere else; inconsistent. Consolidate onto one.
7. **Two vaccine "tables"** — `vaccines` and `vaccine_library` overlap; `Vaccine` is used for logging, `VaccineLibrary` for education; duplicated bilingual fields and seeding. Consider merging or mapping cleanly.
8. **Inconsistent error handling** — some services throw plain `Error` (→ HTTP 500) instead of `NotFoundException/BadRequestException` (e.g. `vet-dashboard.service.ts`, `farmer-detail.service.ts`, `users.service.ts` `updateProfileImage`).
9. **Auth controller ignores its DTOs** — `send-otp.dto.ts/verify-otp.dto.ts` are defined but the controller uses inline `body: { phone: string }`; validation is weaker. `login-response.dto.ts` is an empty file (0 bytes).
10. **Hard-coded JWT secret fallback.**
11. **Public CORS** — `app.enableCors()` without origin allowlist.
12. **Reminder/overdue cron jobs run every minute** (correct but heavy), and responsibilities are split between `reminders` and `notifications`.

● Nice to Improve

13. **Unused frontend dependencies** — `image_picker`, `flutter_secure_storage`, `provider` are in `pubspec.yaml` but never imported/used in `lib`.
14. **Unused backend dependency** — `axios` is in `package.json` but not used (HTTP is via `TypeORM` / no external calls).

15. **Empty / dead folders** — `backend/src/otp`, `backend/src/reminder`, `backend/src/farmer-vet-link`, `backend/src/common` contain no files; `SmsService` is an empty shell.
 16. **Duplicated token-header code** across every Flutter service (a helper would be cleaner).
 17. **Null-safety in Flutter models** — `flock.dart` uses `as int/as String` casts that crash on unexpected JSON (the `sick_report.dart` model is more defensive); `vaccine.dart` defaults are safer.
 18. **Frontend screens duplicate theming/localization maps** — many screens redefine the same colour constants and `_localizedValues` dictionaries.
 19. `auth_service.dart updateProfile()` is an empty stub `({})` — profile update is not implemented.
 20. **PDF export is plain-text named .pdf** — not a real PDF.
 21. **Flutter service methods return loose `List<dynamic>`** (vaccination/reminder) instead of typed models in several places.
-

9. SUPERVISOR QUESTIONS (20, with simple + technical answers + supporting file)

For each: **Simple answer** → **Technical answer** → **File/class/function**.

Q1. Why Flutter?

- Simple: one codebase runs on the farmer's Android app and the vet's device.
- Technical: single Dart UI layer, AOT compilation, web+desktop support, widgets that rebuild on `setState`.
- File: `frontend/lib/main.dart`, `frontend/pubspec.yaml`.

Q2. Why NestJS?

- Simple: a structured, dependable Node.js framework for our REST API.
- Technical: modules/controllers/services, DI, decorators, guards, validation pipes, clean TypeORM integration.
- File: `backend/src/app.module.ts`, `backend/src/main.ts`.

Q3. Why PostgreSQL?

- Simple: a reliable relational database.
- Technical: SQL-standard, enum columns for statuses, JSON support for notifications, mature transactions.
- File: `app.module.ts` TypeORM config, `backend/.env`.

Q4. Why TypeORM?

- Simple: I describe tables as TypeScript classes instead of writing SQL by hand.
- Technical: entity decorators map columns/relations and auto-create the schema.
- File: `users/entities/user.entity.ts`, `flocks/entities/flock.entity.ts`.

Q5. Why JWT?

- Simple: a token that says 'who you are' without a DB lookup each request.

- Technical: stateless issuer { `user_id`, `phone` } with `expiresIn: 7d`, verified via passport-jwt; no server sessions; fits a mobile app.
- File: `auth/auth.service.ts` (sign), `auth/jwt.strategy.ts`, `auth/auth.module.ts`.

Q6. How does OTP work?

- Simple: a 6-digit code is "sent" and the user types it to log in.
- Technical: `send-otp` bcrypt-hashes the code into `OtpCode` with `expires_at`; `verify-otp` compares and issues a JWT. ⚠️ The code today is the dev constant `123456` and is returned in the response; real SMS is not implemented.
- File: `auth/auth.service.ts` `sendOtp/verifyOtp`, `auth/entities/otp-code.entity.ts`, `sms/sms.service.ts` (empty).

Q7. What is a REST API?

- Simple: an API where each URL does a resource job over HTTP.
- Technical: resource endpoints with `GET/POST/PATCH/DELETE`, JSON bodies, stateless.
- File: e.g. `flocks/flocks.controller.ts`.

Q8. What is a Controller?

- Simple: the API's door that receives the request.
- Technical: maps HTTP methods/routes to handlers, reads `@Body()`/`@Param()`, applies `@UseGuards`.
- File: `vaccinations.controller.ts`, `flocks.controller.ts`.

Q9. What is a Service?

- Simple: where the business logic lives.
- Technical: injected classes holding queries and rules, decoupled from HTTP.
- File: `vaccinations.service.ts` (due-date logic), `flocks.service.ts`.

Q10. What is a DTO?

- Simple: the 'shape' of data accepted by an endpoint.
- Technical: validated class (decorators like `@IsInt()`); a global `ValidationPipe` with `whitelist: true` enforces it.
- File: `flocks/dto/create-flock.dto.ts`, `main.ts`.

Q11. What is a Repository?

- Simple: the DB access object for one table.
- Technical: TypeORM `Repository<T>` provides find/save/remove; injected with `@InjectRepository`.
- File: `flocks.service.ts` (`@InjectRepository(Flock)`).

Q12. How are entities related?

- Simple: Users own flocks, flocks have vaccinations, vaccines are on vaccinations, and reminders link to vaccinations.
- Technical: `@OneToMany` on the "one" side, `@ManyToOne` + `@JoinColumn` on the "many" side; `Reminder` uses `onDelete: 'CASCADE'`.
- File: `user.entity.ts`, `flock.entity.ts`, `vaccination.entity.ts`, `reminder.entity.ts`.

Q13. How does Flutter communicate with NestJS?

- Simple: over HTTP with JSON.
- Technical: Flutter services hit `${ApiConfig.baseUrl}`, attaching `Authorization: Bearer <token>` (auto-added by Dio's interceptor in `auth_service.dart`, manual in the `http` services).
- File: `config/api_config.dart`, `services/flock_service.dart`, `services/auth_service.dart`.

Q14. How does the reminder scheduler work?

- Simple: a background task keeps checking due dates.
- Technical: two `@Cron('* * * * *')` methods — `generateReminders` (find `next_due_date = tomorrow`, create pending Reminder), and `generateOverdueNotifications` (overdue → in-app notification). Sending is NOT implemented.
- File: `backend/src/reminders/reminder.scheduler.ts`, `reminders.service.ts`.

Q15. How is the next vaccination date calculated?

- Simple: use the user's date, else add the vaccine's interval to the given date.
- Technical: priority = client `next_due_date` → `date_given` + `interval_days` → `null`. Status via `calculateVaccinationStatus` (`≤3` days = DUE_SOON, past = OVERDUE).
- File: `vaccinations.service.ts` `create()` + `calculateVaccinationStatus()`.
-

Q16. How do you protect farmer data?

- Simple: only the owner sees their own records.
- Technical: JWT guard on routes + ownership checks (flock/vaccination MUST belong to `req.user`), passwords bcrypt-hashed. ⚠ Weaknesses to fix: JWT stored in plaintext, `GET /users` exposes `password_hash`, no role enforcement, fixed OTP.
- File: `flocks.service.ts`, `vaccinations.service.ts`, `users.service.ts`.

Q17. What happens if the API fails?

- Simple: the screen shows an error and lets the user retry.
- Technical: services throw exceptions and use `.timeout(8s)`; UI catches and shows a Retry button. Offline caching is NOT implemented.
- File: `services/*.dart`; e.g. `vaccination_history.dart` error + Retry.

Q18. What happens if there is no internet?

- Simple: the user sees "can't load" and tries again.
- Technical: requests time out and fail with a network error; there's no local cache, so offline data is unavailable.
- File: `flock_service.dart` (`catch` → `Network error`).

Q19. Why both Dio and http?

- Simple: we use Dio just for login (auto token), `http` for the rest.

- Technical: `auth_service.dart` uses Dio with interceptors; all other services use `package:http`. This duplication is inconsistent; I'll consolidate to one client.
- File: `services/auth_service.dart` (Dio), `services/flock_service.dart` (http).

Q20. What technical improvements next?

- Simple: fix the security holes, then make reminders actually send.
- Technical: real OTP/SMS, expiration + attempt limits; JWT in secure storage + role claim + RolesGuard; stop returning password_hash; remove hard-coded secret; real PDF; DB migrations; rate limiting; CORS allowlist; unify HTTP client.
- File: all of the above.

10. ONE-PAGE TECHNICAL SUMMARY (for meeting prep)

What it is: VacTracker is a poultry (chicken) vaccination tracking app. A **farmer** registers chicken flocks, logs vaccinations, sees the vaccine library and reminders, and reports sick birds; a **veterinarian** connects to farmers (by share code), sees their farm health status on a dashboard, and responds to sick reports.

Frontend (Flutter/Dart): One cross-platform UI. Navigation via GoRouter (path-based routes in `main.dart`). API client = `http` for most services + `Dio` (with JWT interceptor) for login. Local persistence = `SharedPreferences` (`StorageService`). Screens are `StatefulWidget`s + `setState` to load and redraw data. PDF "export" shares a plain-text report.

Backend (NestJS/TypeScript): structured into modules → controllers → services → TypeORM repositories → **PostgreSQL**. Features: auth (6-11), users, flocks, vaccines, vaccinations, reminders, sick reports, vet dashboard, notifications, vaccine-library. Global `ValidationPipe` (class-validator) checks DTOs. `@nestjs/config` reads `.env` (DB + JWT).

Auth: phone → check-phone → send-OTP (bcrypt-hashed) → verify → issue JWT (7d) → store → Bearer on requests → `JwtAuthGuard` + `JwtStrategy` → `req.user`.

Main data: User → Flock → Vaccination → Reminder. Each Vaccination links to a Vaccine whose `interval_days` auto-computes the next due date; status = on_time / due_soon ($\leq 3d$) / overdue. Flock → SickReport; User ↔ VetFarmerConnection; Notification.

Reminders: computed at vaccination creation + a `@Cron('* * * * *')` scheduler creates "due tomorrow" reminders; overdue alerts go to the notifications table. The actual sending (SMS/push) is **not yet implemented**.

Key weak spots to mention honestly: real OTP & SMS not done; JWT in plaintext storage; `GET /users` leaks hashes; no real role enforcement; public vaccine endpoints; hard-coded JWT secret; `synchronize: true` in dev; two HTTP clients; empty folders; `image_picker/provider/secure_storage` declared but unused. These are exactly the improvements I propose to tackle next, in §7–§8 of this document.
